

Interview Q&A — Maka Surya Satya Sai Sitaram

Role target: AI Engineer | Full Stack | MLOps & Spring Boot

Based on resume: Maka_Sitaram_AI_Engineer_Resume.pdf

How to use: Read the **short answer** first (30 seconds). Expand with **detailed answer** when interviewer probes. Use **your numbers** (5%+ accuracy, 20% satisfaction, 89% model accuracy, etc.) — they matter.

Table of Contents

- 1. [Opening & General](#)
 - 2. [DOS — LLM & vLLM](#)
 - 3. [DOS — RAG, Agents, Dify, LangChain, MCP](#)
 - 4. [DOS — Embedding Fine-Tuning](#)
 - 5. [DOS — SageMaker, MLOps & Full-Stack AI](#)
 - 6. [DOS — AWS MWAA & Web Scraping Platform](#)
 - 7. [GovTech — Camunda & Offboarding](#)
 - 8. [GovTech — Spring Boot, Kotlin & Integrations](#)
 - 9. [Earlier Roles — FedEx & Hitachi](#)
 - 10. [Key Projects & MSc Thesis](#)
 - 11. [Behavioral & Leadership](#)
 - 12. [Certifications & Skills Validation](#)
-

1. Opening & General

Q: Walk me through your resume / Tell me about yourself.

Short answer (60 sec):

I'm an AI Engineer with 7+ years of software engineering and an MSc in AI/ML. At Singapore's Department of Statistics, I deploy open-source LLMs on vLLM in secure GCC+ environments, build production agents with Dify and LangChain, and fine-tune embedding models for SSIC/SSOC classification — improving accuracy by 5%+ over baseline. Before that at GovTech, I sole-owned the Offboarding module using Camunda 8, Spring Boot, Kotlin, and React. I bridge full-stack engineering, workflow automation, and production ML/MLOps.

Detailed points to hit:

- 7+ years: Hitachi (Camunda/Kafka) → Accolite/FedEx (Java/Angular) → Cognizant/GovTech (Camunda 8, Kotlin) → Cognizant/DOS (LLM, RAG, embeddings, SageMaker)
 - MSc thesis: medical imaging CNNs, BlurPool vs MaxPool
 - Strength: production AI in **government/secure** environments + strong **Java/Spring** foundation
-

Q: Why are you looking for a new role / Why leave DOS?

Short answer:

I've built strong production AI and MLOps experience at DOS and want to apply it at larger scale — either deeper AI product work or a role that combines LLM systems with platform engineering. I'm proud of what we shipped (vLLM serving, embedding fine-tuning, agent frameworks) and I'm ready for the next challenge.

(Adjust honestly to your real situation — never badmouth employer.)

Q: What's your strongest technical area?**Short answer:**

Production LLM systems end-to-end: inference (vLLM), RAG/agents (Dify, LangChain, MCP), embedding fine-tuning (BGE, QLoRA, Ray), plus the full-stack and MLOps to deploy them safely in regulated environments. I'm equally comfortable in Spring Boot/Kotlin when the problem needs robust backend services.

Q: AI Engineer vs Software Engineer — how do you see yourself?**Short answer:**

I'm a software engineer who specializes in AI. I don't just train models — I deploy them: APIs, CI/CD, security (GCC+, SSO), evaluation benchmarks, and integration with business workflows (Camunda, FastAPI, React). That combination is what DOS and GovTech needed.

2. DOS — LLM & vLLM

Q: What did you do with vLLM at DOS?**Short answer:**

I deployed open-source LLMs on the vLLM inference engine in a private GCC+ government cloud. I configured GPU allocation, PagedAttention, continuous batching, and exposed OpenAI-compatible REST endpoints so downstream apps could call models without vendor lock-in.

Follow-up — What is vLLM?

vLLM is a high-throughput LLM **inference** serving framework. It optimizes GPU memory and batching so you can serve large models to many concurrent users with low latency.

Follow-up — What is PagedAttention?

A memory management technique inspired by OS virtual memory. KV cache is stored in non-contiguous "pages" instead of one big contiguous block — reduces memory waste and lets you batch more requests on the same GPU.

Follow-up — What is continuous batching?

Traditional batching waits until all sequences in a batch finish. Continuous batching adds new requests to the batch as soon as a sequence completes — higher GPU utilization and better throughput.

Q: Why open-source LLMs instead of only commercial APIs?**Short answer:**

Government GCC+ environment: data residency, security, cost at scale, and control over model versions. Open-source models on private vLLM give us OpenAI-compatible APIs without sending sensitive statistical data to external providers.

Q: How did you expose models to applications?

Short answer:

OpenAI-compatible REST endpoints — apps can use standard client libraries with a custom base URL. This lowered integration effort for internal teams building RAG and agent workflows.

Q: What challenges did you face deploying LLMs in GCC+?

Short answer (examples — use what actually happened):

- GPU capacity planning and allocation in restricted cloud
- Network/security zones (VPC, PSN subnets)
- Model artifact storage and versioning in secure environments
- Monitoring latency, throughput, and GPU memory under production load
- Coordinating with security around WOG AD and Cognito/SSO

Q: How do you monitor LLM serving in production?

Short answer:

Track request latency (p50/p95), tokens/sec, GPU utilization, queue depth, error rates, and model version. Log prompts/responses carefully with PII redaction in government context. Alert on saturation or elevated latency.

3. DOS — RAG, Agents, Dify, LangChain, MCP

Q: Explain RAG and how you used it.

Short answer:

RAG (Retrieval-Augmented Generation) = retrieve relevant documents first, then pass them to the LLM as context so answers are grounded in your data, not just model memory.

Typical pipeline:

1. User query → embed query
2. Vector search (OpenSearch / similar) over domain docs
3. Top-k chunks + query → LLM → answer
4. Optional: cite sources, guardrails

At DOS: RAG for statistical/domain knowledge — SSIC/SSOC docs, internal guides, agent knowledge bases in Dify.

Q: What is Dify and how did you use it?

Short answer:

Dify is an open-source **LLM application platform** — visual workflows for agents, knowledge bases (RAG), tool calling, and prompt management. I built production agents on Dify and co-led the DOS Tech Drive teaching teams to build agents with RAG, E2B sandbox, and tool routing.

Follow-up — Dify vs LangChain?

Dify	LangChain
Low-code UI, faster for teams	Code-first, maximum flexibility
Built-in knowledge base, ops UI	Compose chains/agents in Python
Good for product/analyst collaboration	Good for custom engineering

I used **both**: Dify for rapid agent delivery; LangChain where we needed custom code.

Q: What is MCP (Model Context Protocol)?

Short answer:

MCP is a standard for connecting LLMs/agents to **tools and data sources** in an interoperable way — like USB-C for AI integrations. I exposed agent capabilities through MCP so different clients and services could use the same tools consistently.

Interview line:

Instead of every agent having custom tool glue code, MCP defines how tools, resources, and prompts are discovered and invoked.

Q: What tools did your Dify agents use? (Tech Drive)

Short answer:

- **Knowledge Base RAG** — internal documents
 - **E2B sandbox** — safe Python code execution
 - **Firecrawl** — web research
 - **Master agent / tool routing** — orchestrate which tool handles which sub-task
-

Q: How do you reduce hallucinations in RAG?

Short answer:

1. Retrieve better chunks (fine-tuned embeddings — my SSIC/SSOC work)
 2. Strong system prompts: "answer only from context"
 3. Low temperature for factual tasks
 4. Citation requirements
 5. Evaluation benchmarks on domain Q&A
 6. Re-ranking retrieved results
-

Q: What is an AI agent vs a chatbot?

Short answer:

A **chatbot** mostly answers from one LLM call (maybe RAG). An **agent** plans steps, calls **tools** (APIs, DB, code sandbox, search), observes results, and loops until the task is done. My Dify/LangChain/MCP work was agent-style: tool routing, RAG, sandbox execution.

4. DOS — Embedding Fine-Tuning

Q: Why did you fine-tune embeddings at DOS?

Short answer:

Generic embeddings (e.g. off-the-shelf BGE) don't understand **Singapore domain language** — SSIC (industry codes) and SSOC (occupation codes). Fine-tuning on domain pairs improved semantic matching for retrieval and classification by **5%+ over baseline** in our benchmarks.

Q: What models did you fine-tune?

Short answer:

BGE and **Gemma** embedding models using Hugging Face Sentence Transformers, PyTorch, QLoRA for parameter-efficient training, and **Ray** for distributed fine-tuning.

Q: Explain MNRL, Triplet Loss, and Sequence Classifier objectives.

MNRL (Multiple Negatives Ranking Loss):

Uses in-batch negatives — one positive pair (query, relevant doc) and other batch items act as negatives. Efficient for retrieval training at scale.

Triplet Loss:

Anchor (query), positive (relevant), negative (irrelevant) — push positive closer, negative farther in embedding space. Classic metric learning.

Sequence Classifier:

Treat matching as classification (match / no-match) on top of encoder — good when you have clear labels.

Short answer:

I evaluated all three on SSIC/SSOC data and picked what performed best on our **custom evaluation benchmarks**, then ensembled top checkpoints.

Q: What is QLoRA?

Short answer:

Quantized Low-Rank Adaptation — fine-tune large models efficiently by:

1. Quantizing base model weights (e.g. 4-bit)
2. Training small low-rank adapter matrices
3. Much less GPU memory → fine-tune bigger models on fewer GPUs

Used for parameter-efficient embedding/LLM fine-tuning at DOS.

Q: Why Ray for fine-tuning?

Short answer:

Ray scales training across multiple GPUs/nodes — distributed data loading, hyperparameter trials, and parallel experiments. Useful when comparing MNRL vs Triplet Loss vs classifier heads on large domain datasets.

Q: What are ensemble methods for embeddings?

Short answer:

Combined top-performing checkpoints (e.g. average embeddings, voting, or learned fusion) to squeeze more accuracy than any single model — part of how we achieved 5%+ gain on SSIC/SSOC tasks.

Q: How did you evaluate embedding quality?

Short answer:

Built **custom benchmarks** aligned to SSIC/SSOC matching tasks — precision@k, recall@k, MRR, or classification accuracy on held-out domain pairs. Compared baseline vs fine-tuned vs ensemble before production promotion.

5. DOS — SageMaker, MLOps & Full-Stack AI

Q: How did you use SageMaker?

Short answer:

Owned training, inference endpoints, and deployment for production ML workloads — model artifacts, endpoint scaling, integration with applications and CI/CD templates.

Q: What CI/CD pipeline templates did you design for AI/ML?

Short answer:

Reusable templates so downstream teams plug in domain logic but share standardized **build, test, deploy** patterns — model validation gates, container builds, deployment to SageMaker or internal infra, security scans, version tagging.

Follow-up — What gates before production?

Data/schema checks, unit tests, model metric thresholds vs baseline, integration tests, approval for promote to prod.

Q: Describe the full-stack AI app you built (React + FastAPI).

Short answer:

React frontend + Python/FastAPI backend supporting:

- Dataset upload
- Exploratory data analysis (EDA)
- ML training workflows
- AI-powered user workflows

Bridge between data scientists and production users in DOS.

Q: PyTorch vs TensorFlow — what did you use when?

Short answer:

PyTorch for research-style work (thesis CNNs, Hugging Face ecosystem, fine-tuning). **TensorFlow** where legacy

pipelines or team standards required it. At DOS both appeared in training/serving paths — I'm comfortable in either; PyTorch is primary for LLM/embedding work.

6. DOS — AWS MWAA & Web Scraping Platform

Q: Tell me about the web scraping platform on AWS MWAA.

Short answer:
I single-handedly delivered a web scraping platform orchestrated with **AWS MWAA (Managed Airflow)** — HA design, VPC with PSN and public subnets, **WOG AD** integration, and **AWS Cognito** for auth.

Q: Why MWAA / Airflow?

Short answer:
Scraping is inherently scheduled, retry-heavy, and DAG-based — fetch, parse, store, validate. Airflow gives observability, dependency management, and backfill. MWAA is managed Airflow on AWS — less ops than self-hosted.

Q: Explain the HA architecture briefly.

Short answer:
Multi-AZ VPC, redundant Airflow components (per MWAA design), durable metadata DB, task retries, idempotent workers, monitoring/alerting. PSN (government network) vs public subnet separation for security compliance.

7. GovTech — Camunda & Offboarding

Q: Tell me about the Offboarding module you owned.

Short answer:
Sole ownership end-to-end at GovTech: Camunda BPMN workflows + Spring Boot/Kotlin REST APIs + React/TypeScript UI for government workforce offboarding — from process design to production.
Why it matters: Shows full-stack + workflow + government domain + independent delivery.

Q: Camunda 7 vs Camunda 8 — differences?

Camunda 7	Camunda 8
Java stack, embedded or shared engine	Cloud-native, Zeebe broker
BPMN 2.0	BPMN 2.0 + new execution semantics
REST API, job executor	gRPC, partitions, exporters

Mature on-prem	Scalable distributed architecture
----------------	-----------------------------------

Short answer:

At GovTech I worked on **both** — Camunda 7 for existing flows; evaluated and deployed **Camunda 8** for complex cases (escalation events, message events, customization). Managed flows via Modeler, Cockpit, Operate.

Q: What BPMN elements did you use in complex cases?

Short answer:

User tasks, service tasks, gateways (exclusive/parallel), **message events**, **escalation events**, timers, subprocesses, boundary events for SLA escalation — especially in Camunda 8 evaluation for government services.

Q: You improved user satisfaction by 20% — how?

Short answer:

BPM 2.0 process redesign on Camunda — clearer task flows, better error handling, faster feedback to users, reduced manual steps. Measured via user satisfaction surveys or service metrics before/after process improvements.

(Be ready to explain what specifically changed in the process.)

Q: What is DMN and did you use it?

Short answer:

DMN (Decision Model and Notation) — table-based business rules (e.g. loan eligibility, routing rules). Used at Hitachi and Camunda projects; pairs with BPMN so decisions are versioned separately from process flow.

8. GovTech — Spring Boot, Kotlin & Integrations

Q: Why Kotlin over Java at GovTech?

Short answer:

Null safety, concise data classes, coroutines potential, team preference on modern GovTech stack — interoperates fully with Java/Spring ecosystem. Same Spring Boot patterns: REST APIs, JPA, testing.

Q: Describe async processing with Lambda, SQS, EventBridge.

Short answer:

Long-running or decoupled work shouldn't block HTTP threads. Spring Boot publishes events → **EventBridge** routes → **Lambda** or consumers process → **SQS** for buffering and retries. Event-driven integration between gov services.

Example flow:

API accepts offboarding request → writes DB → publishes event → async workers send notifications, update external systems.

Q: How did you optimize Excel report generation (1 min → under 20 sec)?

Short answer:

For 8,000+ records — likely combined:

- Streaming/chunked writes instead of loading all in memory
- Reduced redundant DB queries (batch fetch, projections)
- Async generation where appropriate
- Library tuning (e.g. SXSSF for large Excel)
- Caching static reference data

(Prepare your exact technical changes — interviewers will ask.)

Q: Kotlin data migration utility — what did it do?

Short answer:

REST APIs + Kotlin utility to migrate CSV and onboard data into **APEX** — standardized ingestion, validation, 15% cost savings by reducing manual data handling.

Q: React dashboards — RTK Query, OpenSearch, Slack alerts?

Short answer:

Built React/Redux/TypeScript dashboards with **RTK Query** for API caching/fetching. Integrated **OpenSearch** for search/log analytics. Automated **Slack** alerts for 50+ developers using **Mustache** templates for consistent notification formatting.

9. Earlier Roles — FedEx & Hitachi

Q: What did you do at FedEx (Accolite)?

Short answer:

Full-stack pilot banking app — **Java 8**, **Angular 8**, data migration utility (20% accuracy improvement across schemas), PDF/Excel reports, **NgRx** state management.

Q: What did you do at Hitachi Vantara?

Short answer:

Manufacturing/coal mining (Australia):

- **Camunda 7 + Spring Boot** microservices with **Kafka** and **RabbitMQ**
- Analytics framework for data scientists — deploy models via REST + Kafka
- Anomaly detection: code → **decision-based** (DMN/Camunda) + linear regression **89% accuracy**
- **Keycloak SSO**, optimized Kafka consumers, **Docker/Kubernetes**

Award: Inspiration of the Year — HPI project.

Q: How did you optimize Kafka consumers?

Short answer:

Tune `max.poll.records`, `max.poll.interval.ms`, parallel processing with idempotent handlers, proper consumer group sizing, batch processing, avoid long processing in poll loop, monitor lag, use dead-letter topics for poison messages.

Q: Kafka vs RabbitMQ — when did you use which at Hitachi?

Short answer:

Kafka — high-throughput event streaming, analytics pipeline, replay. **RabbitMQ** — task queues, routing to specific workers, traditional messaging. Camunda often integrates with both depending on use case.

10. Key Projects & MSc Thesis

Q: Camunda 8 + SageMaker loan approval project?

Short answer:

Trained **default-prediction model** on SageMaker. **Camunda 8** orchestrates loan approval BPMN — at decision point, call SageMaker endpoint for real-time inference → approve/reject/manual review. Published on Camunda Blog and Medium.

Whiteboard flow:

Application submitted → Camunda process → gather data → SageMaker inference → gateway on score → human task if borderline → outcome.

Q: MSc thesis — BlurPool vs MaxPool?

Short answer:

Compared **BlurPool** (anti-aliasing downsampling) vs standard **MaxPool** in CNNs for **melanoma detection** on medical images. Architectures: ResNet, DenseNet, MobileNet, Inception. BlurPool reduces aliasing when downsampling feature maps — can improve generalization on medical imaging.

Q: How did you handle class imbalance in melanoma detection?

Short answer:

Techniques like weighted loss, oversampling/undersampling, data augmentation, appropriate metrics (AUC, F1, sensitivity/specificity) — medical imaging cares about missing melanoma (false negative) more than false positive.

Q: Medical imaging CNN project on GitHub?

Short answer:

github.com/satyaram413/cnn_melanoma_detection — PyTorch implementations, BlurPool experiments, reproducible training pipeline from MSc work.

11. Behavioral & Leadership

Q: Tell me about a time you worked independently / sole ownership.

Short answer:

GovTech Offboarding module — end-to-end BPMN, backend, frontend.

DOS web scraping platform — single-handed delivery on MWAA.

Shows I can own ambiguous problems without waiting for direction.

Use **STAR**: Situation → Task → Action → Result.

Q: Tell me about teaching / knowledge sharing.

Short answer:

Co-led **DOS Tech Drive** on Dify agents (RAG, E2B, tool routing) — DOS-wide session, well received. At GovTech, knowledge-transfer sessions on Camunda to engineering teams. Collaborated with Camunda Technical Associates.

Q: Tell me about a difficult production issue.

Prepare one real story:

e.g. embedding model regression after fine-tune, vLLM GPU OOM under load, Camunda incident stuck workflow, Kafka consumer lag. Structure: detect → mitigate → root cause → prevent recurrence.

Q: How do you work with non-technical stakeholders?

Short answer:

DOS Tech Drive is an example — hands-on sessions for building agents, not just slides. Translate technical trade-offs (accuracy vs latency vs cost) into business terms. SSIC/SSOC work tied directly to classification accuracy stakeholders care about.

Q: SAFe Scrum Master certification — how do you use it?

Short answer:

Facilitate ceremonies, remove blockers, align teams on increments. In government projects with multiple vendors/stakeholders, structured agile helps deliver AI and workflow features predictably.

12. Certifications & Skills Validation

Q: OCI Generative AI Professional — what did you learn?

Short answer:

Oracle Cloud Gen AI services, model deployment on OCI, RAG patterns, security and integration on OCI — complements hands-on DOS private-cloud LLM work.

Q: Azure AI Fundamentals?

Short answer:
Foundational ML/AI concepts on Azure — Cognitive Services, Azure ML basics, responsible AI. Breadth certification; DOS work was heavier on AWS/OCI/private cloud.

Q: Hugging Face Agents course?

Short answer:
Agent architectures, tool use, HF ecosystem — directly applied in LangChain/Dify/MCP agent work at DOS.

Q: Redis RU101?

Short answer:
Redis data structures — strings, hashes, lists, sets, sorted sets. Useful for caching embeddings, session state, rate limiting in API/LLM serving layers.

Q: How do you stay current in AI?

Short answer:
Hands-on projects (DOS production), HF/OCI courses, Camunda/blog publications, GitHub (melanoma CNN), internal tech drives, following inference (vLLM), RAG, and agent frameworks (MCP, Dify).

Quick-Fire Round (rapid answers)

Topic	One-line answer
vLLM	High-throughput LLM inference with PagedAttention + continuous batching
RAG	Retrieve docs → augment prompt → generate grounded answer
MCP	Standard protocol for LLM tool/data integration
QLoRA	Efficient fine-tuning with quantized base + low-rank adapters
MNRL	In-batch negative ranking loss for retrieval training
SSIC/SSOC	Singapore industry/occupation codes — domain embedding fine-tune
Camunda 8	Cloud-native Zeebe-based BPMN engine
GCC+	Singapore government commercial cloud plus security controls
MWAA	Managed Apache Airflow on AWS

BlurPool	Anti-aliased pooling for CNN downsampling
Ensemble embeddings	Combine best checkpoints for higher retrieval accuracy

Questions YOU should ask the interviewer

- 1. What does the ML/LLM stack look like today — training vs inference split?
- 2. How do you evaluate RAG/agents before production?
- 3. Team balance: research vs platform vs product engineering?
- 4. Cloud and data residency requirements?
- 5. Biggest bottleneck: model quality, latency, cost, or compliance?

Document generated from resume: Maka_Sitaram_AI_Engineer_Resume.pdf
Practice aloud — short answer first, then depth.